

Fuel Platform — Production Architecture

Goal: one codebase, one database, that a client can pick up in one of two modes at provisioning time:

- **Mode A — Full product:** Ordering (marketplace) + Monitoring + Delivery proof.
- **Mode B — Monitoring-only:** Monitoring + Delivery proof, fed by the client's own ordering system over a Dispatch API.

The whole architecture is organized around one rule from the product doc: **separate the commercial `Order` from the physical `Trip`. Monitoring runs on `Trip`. A per-tenant module switch decides whether the Ordering workflow exists.**

Assumptions (stated so you can override): multi-tenant SaaS; Galileosky hardware for fuel + GPS; ~hundreds of trucks per tenant growing to thousands; Postgres-centric. Swap specifics freely — the shape holds.

1. The one decision that drives everything: the module switch

A tenant is provisioned with a module configuration. **This single object is the "two products" mechanism** — there is no fork, no second deployment.

```
json
// tenant.modules
{
  "monitoring": true,    // always on — it's the core product
  "ordering":  false,   // Mode A = true, Mode B = false
  "dispatchApi": true   // Mode A = optional, Mode B = true
}
```

This config is read by **three** layers, and that's the entire trick:

Layer	What it gates
AuthZ middleware	Whether <code>place_order</code> , <code>accept_order</code> , <code>assign_transporter</code> , <code>kyc_*</code> permissions are <i>effective</i> for any role
API gateway / router	Whether Ordering endpoints and the Dispatch endpoint are mounted for this tenant
Frontend	Which menu items / screens render (the roles are identical; menus light up differently)

```
effective_permission(user, action, tenant) =  
    role_grants(user, action)           // does the role allow it?  
    AND module_enabled(tenant, action)   // is the owning module on for  
    this tenant?
```

Turn `ordering` off and the marketplace permissions become inert for *every* role automatically — no role edits, exactly as the doc requires.

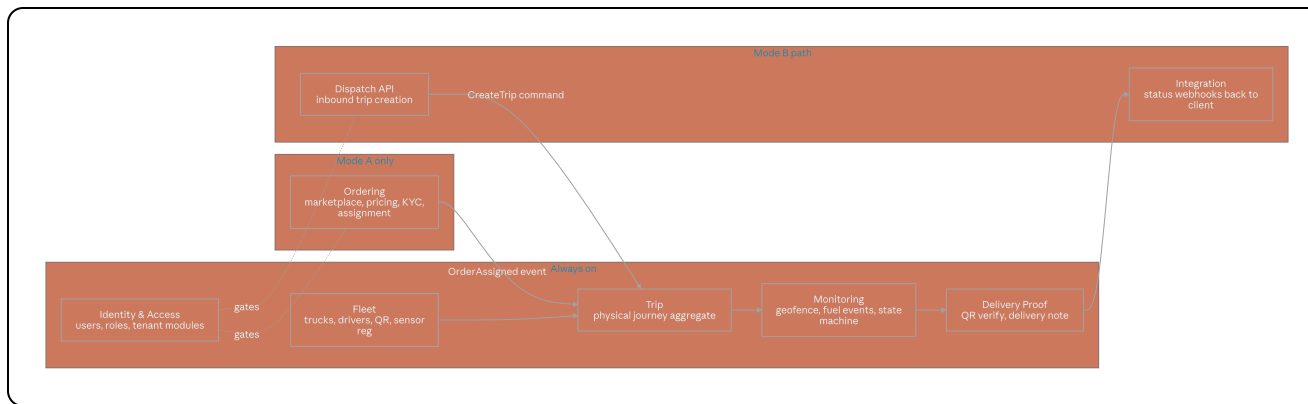
2. System context



Telemetry ingestion is a **separate process** from the app core because it has a totally different runtime profile (long-lived TCP sockets, high-frequency packets). Everything else lives in one deployable.

3. Bounded contexts (modules inside one codebase)

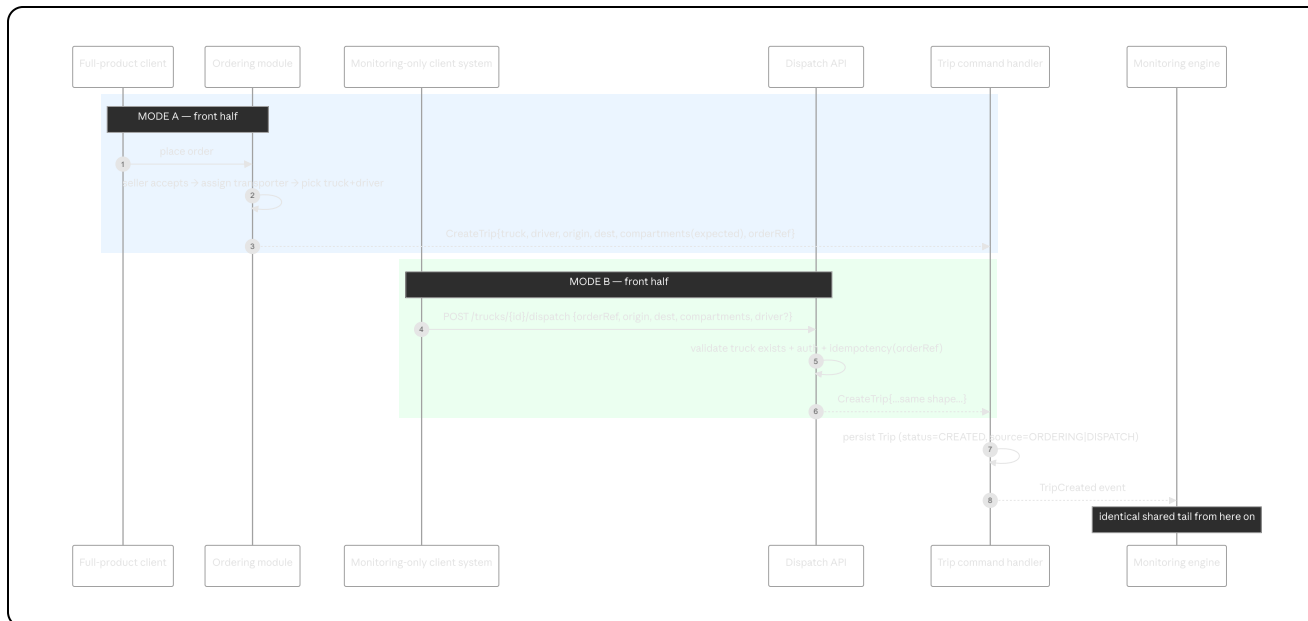
A **modular monolith** is the right call here — the doc explicitly wants one codebase/DB, and these contexts share the `(Trip)` aggregate heavily. Keep hard module boundaries so you *could* extract later, but don't pay the distributed-systems tax now. The one exception already split out is the Telemetry Gateway.



The seam is **Trip**: both **Ordering** and **Dispatch** converge on the same **CreateTrip** command, and the monitoring half never learns which door the trip came through.

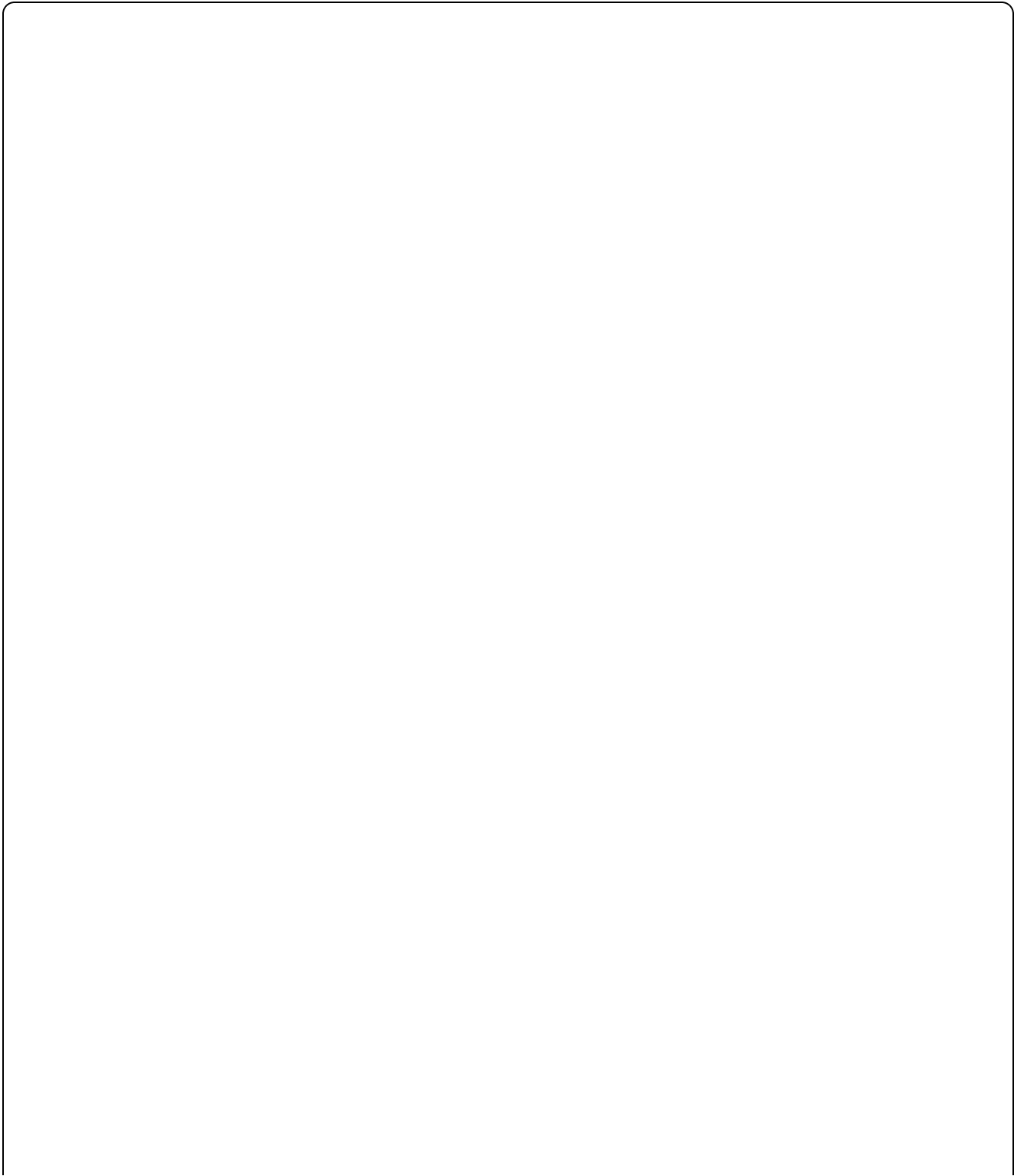
4. The Trip-creation seam (where the two modes meet)

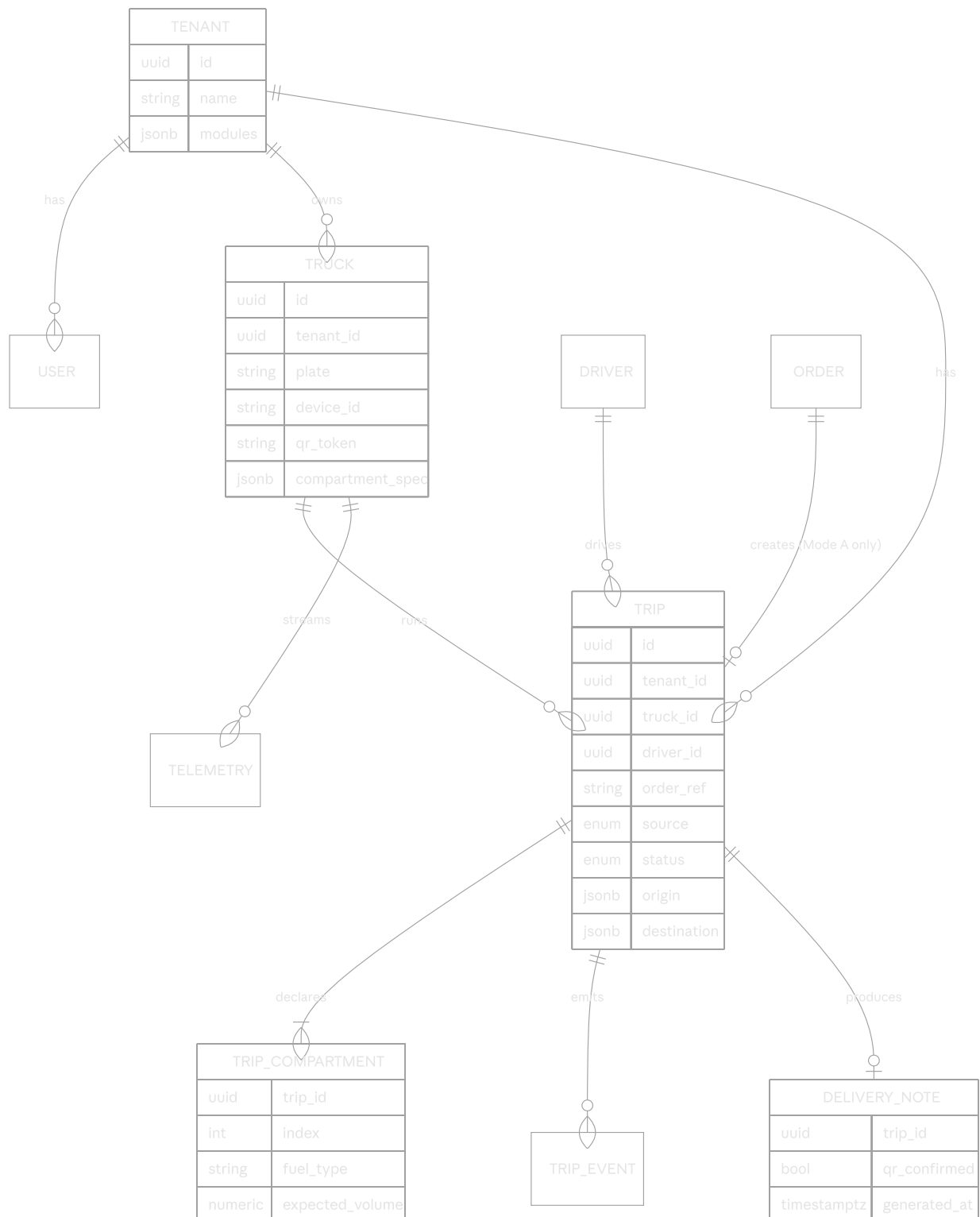
This is the architectural heart. Both modes produce an identical **Trip** aggregate via one command handler.



The `CreateTrip` command is the contract. Whatever fills it — an internal marketplace event or an external HTTP POST — is irrelevant downstream.

5. Domain & data model





numeric	loaded_volume
numeric	delivered_volume

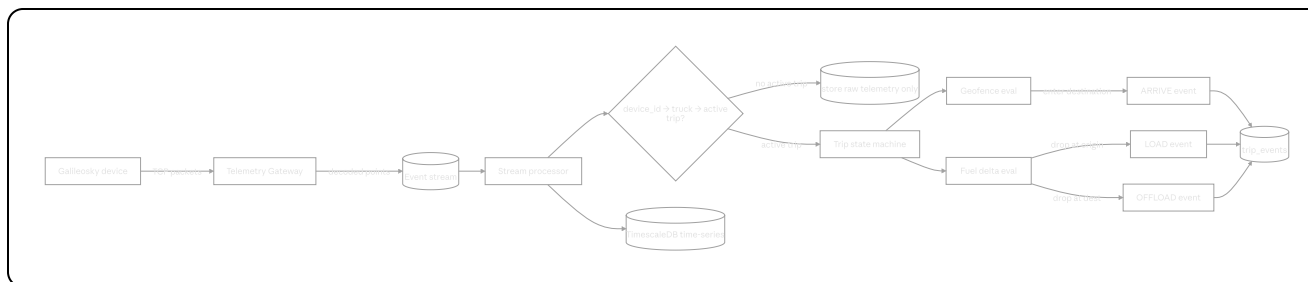
string	pdf_url
--------	---------

Key modeling choices:

- **trip.order_ref** is a **plain string**, not a hard FK. In Mode A it mirrors the internal order id; in Mode B it's the client's own order number. This keeps **Trip** decoupled from **Order** — **Order** can be absent entirely.
- **trip.source = ORDERING | DISPATCH** records the door for analytics/debugging only; no logic branches on it after creation.
- **ORDER** and its **satellite tables** (offers, assignments, KYC) physically exist in the schema but are simply never written for Mode B tenants. Same DB, unused rows.
- **compartment** carries the **three numbers** that make the delivery note: **expected** (the promise from order/dispatch), **loaded** (sensor at origin), **delivered** (sensor after offload).

6. Monitoring pipeline (the high-throughput half)

This is where most production engineering effort goes. It is identical for both modes.



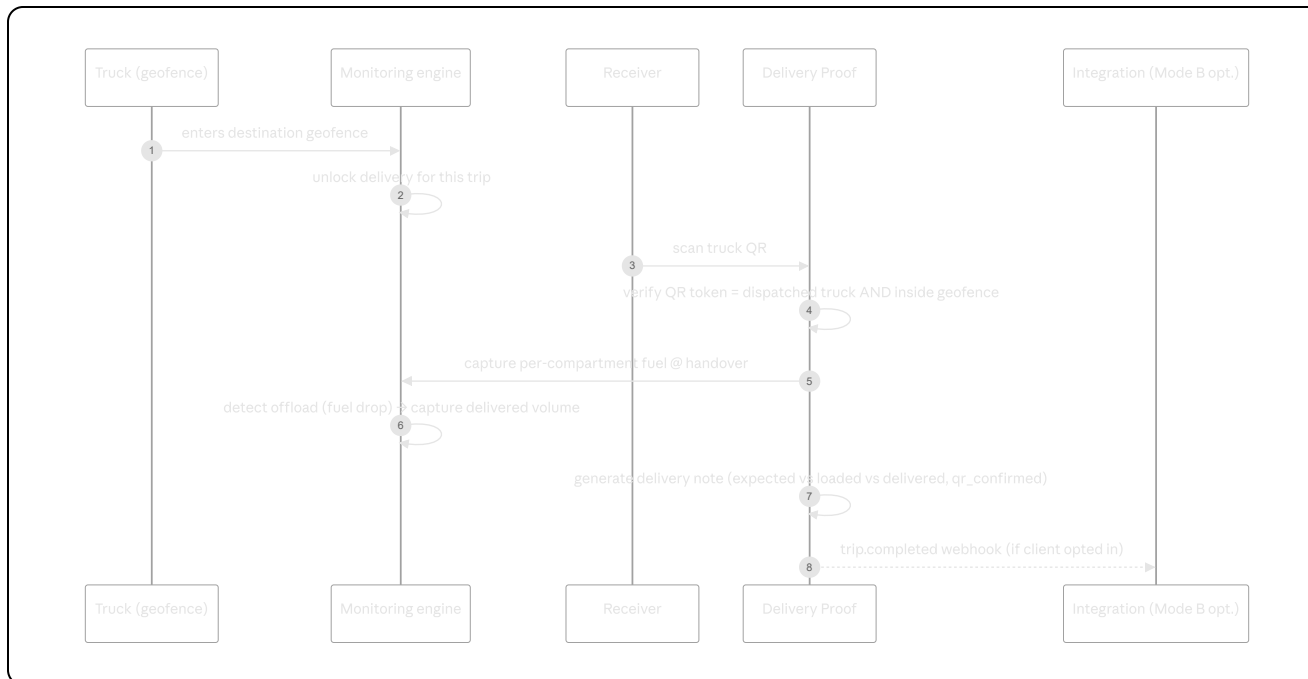
Components:

- **Telemetry Gateway** — a dedicated service (Go or Node) that speaks the Galileosky binary protocol over raw TCP, holds long-lived sockets, decodes packets, and

publishes normalized points to the stream. Scale it by **sharding devices across instances** (consistent hashing on `device_id`); it's the only stateful-connection tier.

- **Stream** — Kafka / NATS / Redis Streams. Decouples ingestion spikes from processing and gives you replay.
 - **Stream processor** — resolves `device → truck → currently-active trip`, writes raw points to TimescaleDB, and feeds the per-trip state machine.
 - **Trip state machine** — drives the shared tail: `CREATED → LOAD → EN_ROUTE → ARRIVE → QR_SCAN → OFFLOAD → RETURN → COMPLETED`. Transitions come from fuel deltas and geofence crossings, not manual timers.
 - **TimescaleDB** — telemetry is time-series and high-volume; keep it out of the core Postgres tables and use hypertables + retention/continuous-aggregates.
-

7. Delivery proof flow (shared tail)

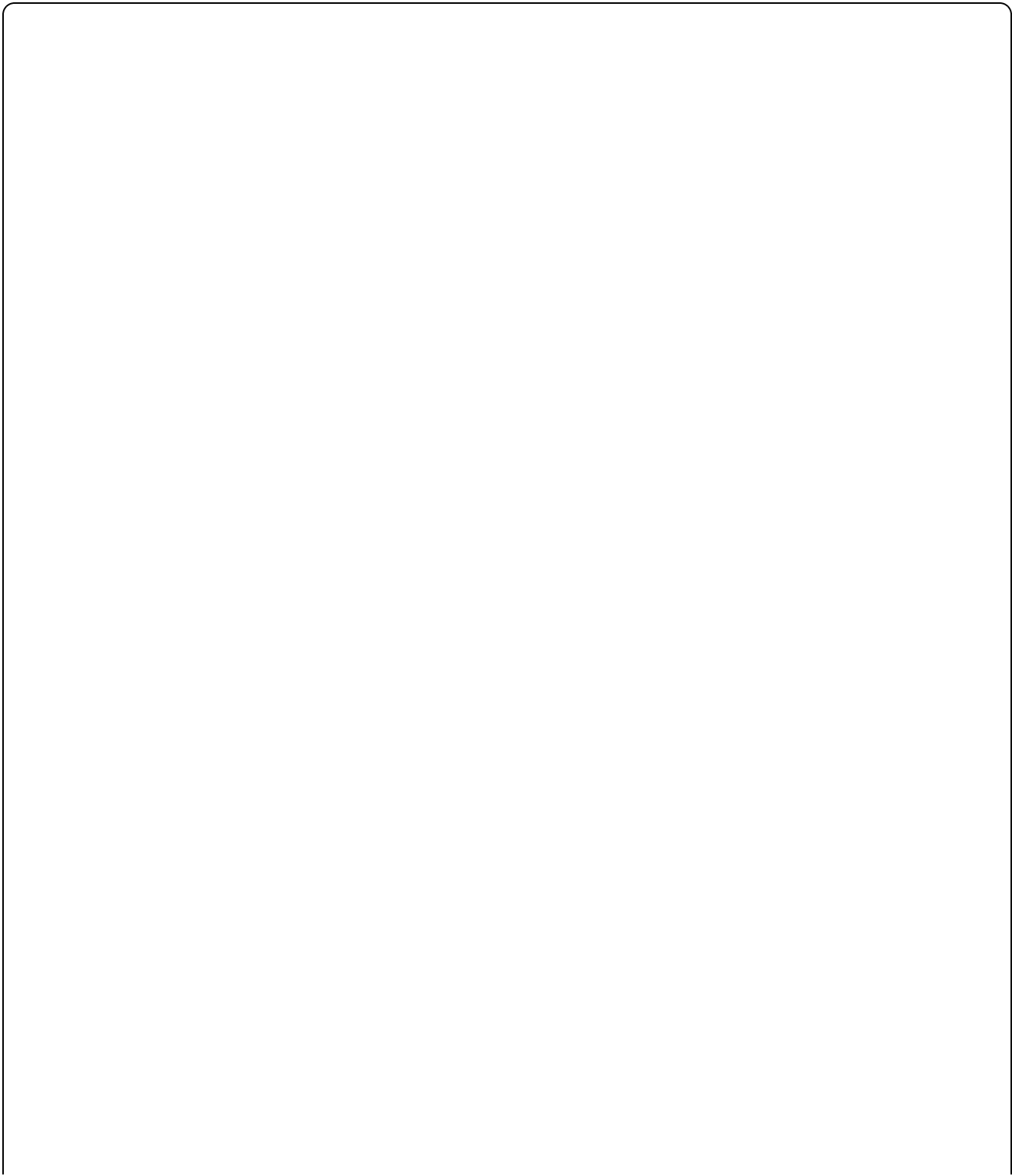


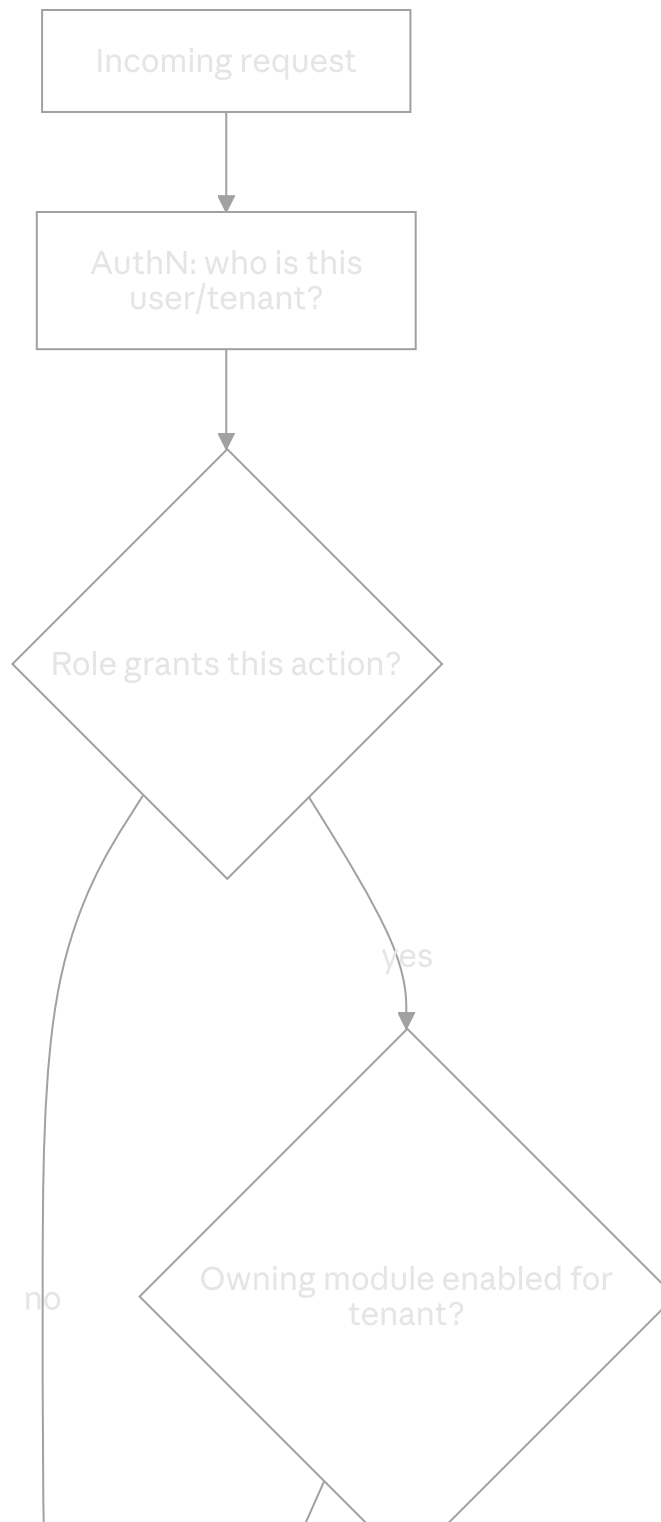
QR verification is a security check, not a convenience: the scanned token must map to *the truck dispatched for this specific trip*, and the truck must be inside the destination geofence. That's what defeats a lookalike truck.

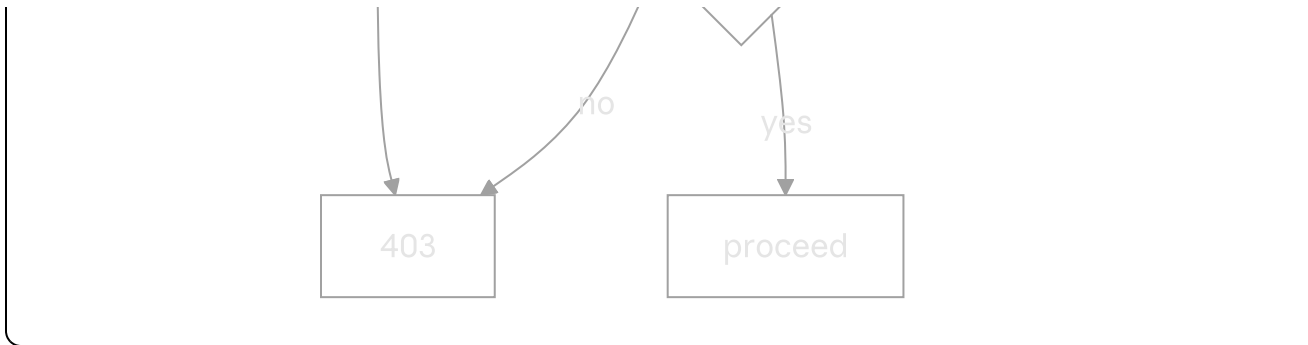
The **delivery note** is the product's whole value: a per-compartment expected vs loaded vs delivered table backed by sensor data, plus the qr_confirmed flag, order ref, timestamps. Render to PDF, store in object storage, expose URL.

8. Roles & access control

Roles are **fixed and identical across modes** (Platform Admin, Transport Admin, Driver, Client/Receiver, Seller Manager). Mode differences come *only* from the module gate.







Worked example: `Seller Manager` keeps the `accept_order` grant in Mode B, but `ordering` is off for that tenant, so the gate denies it and the screen never renders. No role was renamed or removed — exactly the doc's promise.

Implementation: permission → owning-module map, evaluated in one authorization middleware. Cache the tenant `modules` object in Redis; it changes rarely.

9. API surface (mode-aware)

Endpoint group	Mode A	Mode B	Auth
<code>/orders/*</code> , <code>/offers/*</code> , <code>/kyc/*</code>	mounted	not mounted	user session (JWT)
<code>/trucks/*</code> , <code>/drivers/*</code> , <code>/trips/*</code> (read), QR scan	mounted	mounted	user session
<code>POST /trucks/{id}/dispatch</code>	optional	mounted	machine token (API key / OAuth client-credentials)
<code>POST /webhooks</code> (outbound to client)	—	optional	signed payloads (HMAC)

The **Dispatch endpoint** is a machine-to-machine API, authenticated differently from the human UI:

- **Idempotent** on `(tenant_id, orderRef)` — a retried dispatch must not create a second trip.
- **Validates the truck exists** and belongs to the tenant before creating a trip (the doc's non-negotiable #1).
- Returns `409` if `orderRef` already mapped to a trip; `404` if truck unknown.
- Rate-limited and per-client key-scoped.

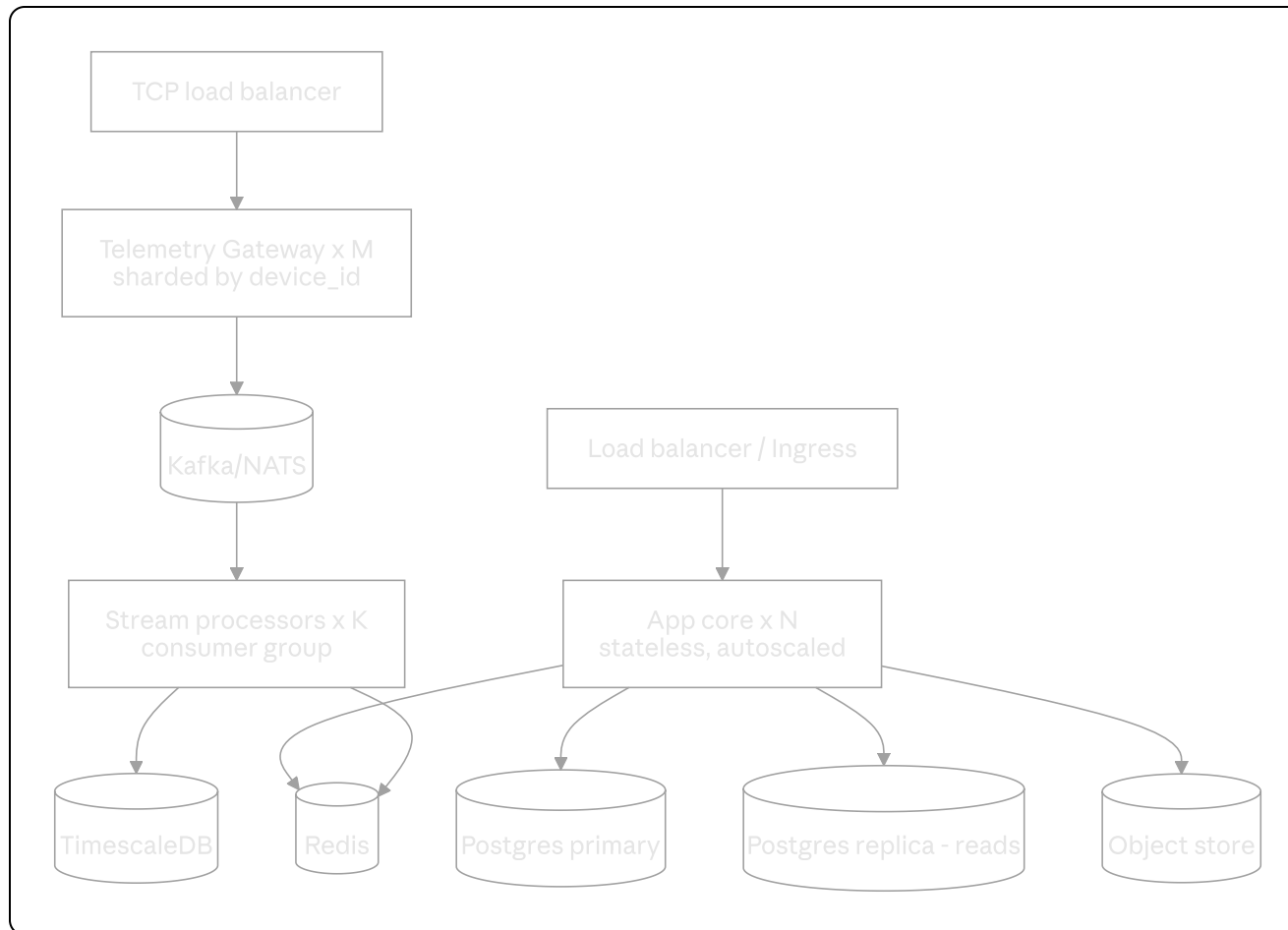
Outbound webhooks (status push back to the client's order system) need retries with exponential backoff, a dead-letter queue, and HMAC signatures so the client can verify authenticity. Make this opt-in per tenant.

10. Recommended stack

Concern	Recommendation	Why
Core app	Modular monolith — NestJS (TS) or Spring Boot (Java) or Go	Doc mandates one codebase/DB; strong module boundaries
Core DB	PostgreSQL (+ Row-Level Security for tenancy)	Relational domain, transactional ordering workflow
Telemetry store	TimescaleDB (Postgres extension)	Time-series fuel/GPS, hypertables, retention
Telemetry gateway	Dedicated Go/Node TCP service	Galileosky binary proto, long-lived sockets

Concern	Recommendation	Why
Stream	Kafka (scale) or NATS/Redis Streams (lean)	Decouple ingest from processing; replay
Cache	Redis	Tenant module config, active-trip lookup, sessions
Object storage	S3-compatible	Delivery-note PDFs
Frontend	React, role-and-module-aware menu	Same screens, gated rendering
Infra	Kubernetes (or ECS)	Scale app and gateway tiers independently
Observability	OpenTelemetry + Prometheus/Grafana + structured logs	Trip-level tracing across the pipeline

11. Deployment topology



- **App core** is stateless → scale horizontally behind the LB.
 - **Telemetry gateway** is the only tier with sticky state (open sockets) → scale by device sharding, not round-robin.
 - **Stream processors** scale as a consumer group on partition count.
 - **Postgres** primary for writes, replica for the heavy read/reporting side (live fleet maps, history).
-

12. Cross-cutting concerns

- **Multi-tenancy isolation** — every row carries `(tenant_id)`; enforce with Postgres Row-Level Security so a query can't leak across tenants even with a bug. Telemetry partitioned by tenant too.
 - **Idempotency** — Dispatch API keyed on `((tenant, orderRef))`; webhook deliveries keyed on event id. Both retry-safe.
 - **Security** — human UI on JWT/session; Dispatch + webhooks on machine credentials; QR tokens are signed, single-truck-scoped, and only valid inside the active trip's geofence window; consider mTLS on the device gateway if hardware supports it.
 - **Observability** — propagate a `(trip_id)` as the trace correlation id from sensor packet → state machine → delivery note, so one trip is one searchable trace.
 - **Data retention** — raw telemetry is huge; use TimescaleDB continuous aggregates for dashboards and drop raw points after N days, keeping `(trip_events)` + `(delivery_note)` forever (they're the legal proof).
-

13. Build & rollout sequence

The doc's "new work is small and additive" maps to this order. Each step ships independently and doesn't disturb existing clients.

1. **Introduce `(Trip)` as a first-class aggregate**, internally separate from `(Order)`. Migrate existing flows so `(Order)` creates a `(Trip)` rather than being the trip. (Pure refactor; behavior unchanged for current full-product clients.)
2. **Add the `(tenant.modules)` switch + the authZ/router/UI gating**. Existing tenants get `{ordering:true}` — no visible change.
3. **Build the Dispatch API** (`(POST /trucks/{id}/dispatch)`) with truck-existence validation + idempotency, emitting the same `(CreateTrip)` command.

4. **Add the manual "create trip" screen** for monitoring-only clients with no API of their own (same command, human-entered).
5. **Add outbound status webhooks** (opt-in) for clients who want their order auto-closed.
6. **Onboard the first monitoring-only tenant** with `{ordering:false, dispatchApi:true}` and validate the shared tail end-to-end.

Steps 1-2 are the only ones that touch existing code paths, and both are designed to be no-ops for current behavior. Everything after is purely additive surface area.

14. One-screen summary

- **Two products = one tenant flag.** `ordering` on/off; nothing forks.
- **`Trip` is the universal seam.** Order-driven creation and Dispatch-driven creation both emit the same `CreateTrip`; monitoring is blind to the source.
- **Roles never change.** A permission is effective only if `role grants it AND its module is on` — turning ordering off silently hides the marketplace for every role.
- **The expensive, shared core (telemetry → geofence → QR → delivery note) is built once** and runs identically in both modes; it's deliberately split into its own ingestion tier for scale.
- **New build is additive:** Trip/Order split, Dispatch API, manual trip screen, webhooks — none disturbs existing full-product clients.